

IDENTIFICACIÓN DEL PROYECTO

NOMBRE DEL PROYECTO: Detección de Flotas Logísticas mediante Clasificación Vehicular con CNN

TIPO DE PROYECTO: Aplicación de Deep Learning — Visión por Computadora

ESPACIO CURRICULAR: Procesamiento de Imágenes **DOCENTE:** Carlos Ignacio CHARLETTI

INTEGRANTES: Mario Arce | Ariel Denaro | Simón Vottero

AÑO: 2026

Consigna Evidencia 2: "Implementación inicial del proyecto aplicando técnicas de procesamiento de imágenes y registro de avances del proyecto."

Colab:  `ev2_veh_classifier.ipynb`

Repositorio: https://github.com/arielden/ISPC_ProcImage_VehicleClass

1. Resumen de Avances

En esta segunda evidencia se documenta la implementación inicial del pipeline de datos y el primer modelo de clasificación funcional. El notebook `ev2_veh_classifier.ipynb` contiene un flujo completo que abarca desde la carga y filtrado del dataset CIFAR-10 hasta la obtención de predicciones individuales sobre imágenes de prueba.

Como punto de partida, se optó por construir una red neuronal densa en lugar de una CNN, mediante esta estrategia, establecemos una **línea base de rendimiento (baseline)** antes de introducir la complejidad de capas convolucionales. Justificamos esta decisión en la sección 4.

OE#	Objetivo Específico	Estado	Avance
OE1	Analizar y preprocesar el dataset	COMPLETADO	100 %
OE2	Aplicar técnicas de procesamiento de imágenes	EN CURSO	40 %
OE3	Diseñar y entrenar el modelo CNN	BASELINE ACTIVO (modelo de base)	25 %
OE4	Evaluar el rendimiento del modelo	PARCIAL	25 %
OE5	Cuantificar el impacto logístico	PENDIENTE	0 %
OE6	Informe final	PENDIENTE	0 %

2. Pipeline de Datos Implementado

2.1 Carga del Dataset y Filtrado de Clases

Cargamos el dataset CIFAR-10 mediante “tensorflow_datasets”, que provee los splits `train` (50.000 imágenes) y `test` (10.000 imágenes). Se conservan únicamente las dos clases de interés:

```
# Clases de interés
CLASE_LIVIANO = 1 # automobile
CLASE_PESADO = 9 # truck

# Filtrado: se descartan las otras 8 clases
def filtrar_vehiculos(imagen, etiqueta):
    return tf.logical_or(
        tf.equal(etiqueta, CLASE_LIVIANO),
        tf.equal(etiqueta, CLASE_PESADO))

# Reasignación binaria: 0 = Liviano, 1 = Pesado
def remap_etiquetas(imagen, etiqueta):
    nueva_etiqueta = tf.cast(tf.equal(etiqueta, CLASE_PESADO), tf.int64)
    return imagen, nueva_etiqueta
```

El filtrado conserva **10.000 imágenes de entrenamiento** (5.000 por clase) y **2.000 de prueba** (1.000 por clase), garantizando un dataset perfectamente balanceado que elimina la necesidad de de oversampling.

2.2 Normalización de Imágenes

Se aplica normalización Min-Max al rango [0, 1] sobre los valores de píxeles [0, 255]. Esta es la técnica de procesamiento de imágenes fundamental con la que garantizamos la estabilidad numérica del entrenamiento:

```
def normalizar(imagenes, etiquetas):
    imagenes = tf.cast(imagenes, tf.float32)
    imagenes /= 255.0 # Rango [0, 255] → [0.0, 1.0]
    return imagenes, etiquetas
```

Justificación técnica: Sin normalización, los gradientes del descenso de gradiente estocástico (SGD/Adam) serían desiguales entre distintas regiones del espacio de entrada, ralentizando la convergencia. La normalización centra la distribución de activaciones y estabiliza el aprendizaje.

2.3 Visualización Exploratoria del Dataset

Se implementaron dos niveles de visualización para validar la integridad del pipeline antes del entrenamiento:

Visualización individual	Grilla 5×5 (25 muestras)
<pre>datos_entrenamiento_raw.take(1)</pre> Muestra una imagen aislada con su etiqueta remapeada (Liviano/Pesado) y colorbar de intensidad para verificar el rango de valores.	<pre>datos_entrenamiento_raw.take(25)</pre> Grilla 5×5 con las primeras 25 imágenes del dataset filtrado. Permite detectar visualmente si el filtrado funcionó correctamente y si las etiquetas son coherentes.

2.4 Preparación de Lotes de Entrenamiento

La preparación de los datos sigue el pipeline recomendado por TensorFlow para conjuntos de datos en memoria, separamos las imágenes en batches de entrenamiento:

```
ej_entrenamiento = 10000 # total de imágenes de entrenamiento
ej_pruebas       = 2000  # total de imágenes de prueba
LOTE              = 32    # batch size

datos_entrenamiento = (
    datos_entrenamiento_raw
```

```

        .map(normalizar)          # Normalización
        .cache()                  # Caché en RAM (evita re-lectura)
        .shuffle(ej_entrenamiento) # Aleatorización
        .batch(LOTE)              # Mini-lotes de 32
        .repeat()                 # Repetición indefinida para epochs
    )

datos_pruebas = (
    datos_pruebas_raw
    .map(normalizar)          # Normalización (sin shuffle ni repeat)
    .cache()
    .batch(LOTE)
)

```

Decisión técnica — cache(): Al ser CIFAR-10 un dataset de tamaño manejable, el uso de `.cache()` almacena los datos normalizados en RAM tras la primera época, eliminando I/O de disco en las épocas siguientes y acelerando significativamente el entrenamiento.

3. Técnicas de Procesamiento de Imágenes Aplicadas

3.1 Técnicas Implementadas en esta Entrega

Técnica	Descripción
Normalización de píxeles	Escala lineal [0,255] → [0.0,1.0] por división escalar. Fundamental en cualquier pipeline de ML para procesamiento de imágenes.
Visualización de canales RGB	Despliegue de imágenes 32×32×3 mediante <code>matplotlib.imshow()</code> . Permite verificar la distribución de color y la coherencia del etiquetado.
Etiquetado binario	Reasignación de etiquetas originales CIFAR-10 {1, 9} al esquema binario {0=Liviano, 1=Pesado} mediante operación vectorial TensorFlow.
Shuffle	Aleatorización del orden de las imágenes en cada época para evitar sesgos por el orden natural del dataset.

3.2 Técnicas Planificadas para la Evidencia 3

En la próxima evidencia introduciremos las técnicas de detección de bordes y morfología que serán incorporadas como **etapa de extracción de características** previa al modelo CNN. Se planifica su implementación de la siguiente manera:

Técnica	Aplicación prevista en el proyecto	API / Función
Operador Sobel	Realce de bordes de vehículos (siluetas de camión vs. automóvil) como canal adicional de entrada al modelo.	<code>cv2.Sobel(img, cv2.CV_64F, 1, 0)</code>
Algoritmo de Canny	Detección de contornos finos de vehículos para análisis morfológico comparativo entre clases Liviano y Pesado.	<code>cv2.Canny(img, minVal, maxVal)</code>

Filtro Laplaciano (LoG)	Segunda derivada para detección isotrópica de bordes en las imágenes 32×32 del dataset.	<code>cv2.Laplacian(img_blur, cv2.CV_64F)</code>
Data Augmentation	Rotaciones, flip horizontal y ajuste de brillo para ampliar efectivamente el dataset y mejorar la generalización.	<code>tf.keras.layers.RandomFlip, RandomRotation</code>

4. Modelo de Clasificación Inicial (Red Densa — Baseline)

4.1 Fundamento de la Elección de esta arquitectura

El modelo implementado en esta entrega es una **red neuronal densa**, no una red convolucional. Esta decisión responde a una práctica estándar en el desarrollo de modelos de ML:

Por qué iniciamos con una red densa?	Limitación respecto de la CNN objetivo
Simplicidad verificable: una red funcional, sin bugs, confirma que el pipeline de datos funciona correctamente antes de añadir complejidad propia de una CNN. Métricas de referencia: el accuracy de la red densa, define el umbral mínimo que la CNN deberá superar para justificar su mayor costo computacional. Diagnóstico de datos: si la red densa no converge, el problema está en los datos, no en la arquitectura convolucional.	Sin invariancia espacial: el Flatten() destruye la estructura 2D de la imagen; el modelo no puede detectar patrones locales (bordes, texturas). Sin jerarquía de características: no aprende bordes → formas → estructuras como lo hace una CNN. Rendimiento acotado: se espera un accuracy < 80% en este dataset, confirmando la necesidad de la arquitectura CNN en la entrega siguiente.

4.2 Arquitectura del Modelo

```

modelo = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(32, 32, 3), dtype=tf.float32),
    # → (32, 32, 3) = 3.072 valores por imagen

    tf.keras.layers.Flatten(),
    # → (3072,) - destruye estructura espacial 2D

    tf.keras.layers.Dense(128, activation=tf.nn.relu),
    # → (128,) - primera capa oculta, 393.344 parámetros

    tf.keras.layers.Dense(128, activation=tf.nn.relu),
    # → (128,) - segunda capa oculta, 16.512 parámetros

    tf.keras.layers.Dense(2, activation=tf.nn.softmax)
    # → (2,) - salida: [P(Liviano), P(Pesado)]
])

```

Capa	Forma de salida	Parámetros	Activación
Input + Flatten	(3.072)	0	—
Dense (128 neuronas)	(128)	393.344	ReLU

Dense (128 neuronas)	(128)	16.512	ReLU
Dense — Salida (2 clases)	(2)	258	Softmax
TOTAL	—	410.114	—

4.3 Compilación y Configuración de Entrenamiento

```

modelo.compile(
    optimizer='adam',
    loss=tf.keras.losses.SparseCategoricalCrossentropy(),
    metrics=['accuracy']
)

historial = modelo.fit(
    datos_entrenamiento,
    epochs=10,
    steps_per_epoch=math.ceil(ej_entrenamiento / LOTE) # 313 pasos/época
)

```

Hiperparámetro	Valor y justificación
Optimizador	Adam — adaptativo, robusto, estándar para clasificación de imágenes.
Función de pérdida	SparseCategoricalCrossentropy — trabaja directamente con etiquetas enteras (0 y 1) sin necesitar one-hot encoding, eficiente para clasificación multiclase con pocas categorías.
Batch size	32 — balance entre velocidad de actualización y estabilidad del gradiente. Valor de referencia en la literatura de Deep Learning.
Épocas	10 — suficiente para observar convergencia en el modelo sobre el CIFAR-10 filtrado. La CNN de la próxima entrega usará early stopping para evitar overfitting.

5. Resultados Preliminares y Visualizaciones

5.1 Curvas de Entrenamiento

Se grafican las curvas de **pérdida (loss)** y **precisión (accuracy)** por época para monitorear la convergencia del modelo. Las curvas permiten detectar problemas como sobreajuste (overfitting) o divergencia antes de la evaluación formal:

```

plt.subplot(1, 2, 1)
plt.plot(historial.history['loss'])
plt.title('Pérdida durante el entrenamiento')

plt.subplot(1, 2, 2)
plt.plot(historial.history['accuracy'])
plt.title('Precisión durante el entrenamiento')

```

Comportamiento esperado del modelo: dado que el modelo aplana la imagen (Flatten), pierde la estructura espacial. Se espera una convergencia a un accuracy en el rango **65%–75%**, lo que confirma la necesidad de pasar a una arquitectura CNN que preserve y explote la estructura 2D de las imágenes.

5.2 Evaluación sobre el Conjunto de Prueba

```
perdida, precision = modelo.evaluate(datos_pruebas)
print(f'Pérdida en pruebas: {perdida:.4f}')
print(f'Precisión en pruebas: {precision*100:.2f}%')
```

5.3 Visualización de Predicciones

Se implementó una grilla 5×5 de predicciones (25 imágenes) con barras de confianza por clase. Las predicciones se muestran en **azul** si son correctas y en **rojo** si son incorrectas, junto con el porcentaje de confianza:

```
# Para cada imagen: panel izquierdo = imagen + etiqueta predicha
#                               panel derecho = barras de probabilidad [Liviano | Pesado]
plt.suptitle('Predicciones (Azul=correcto, Rojo=incorrecto)')
```

5.4 Predicción sobre Imagen Individual

Se implementó la función de inferencia sobre una imagen aislada, validando el flujo completo de inferencia end-to-end:

```
imagen_suelta = np.array([imagenes_prueba[imagen_idx]])
prediccion     = modelo.predict(imagen_suelta)

print('Imagen real:      ', nombres_clases[etiquetas_prueba[imagen_idx]])
print('Predicción:      ', nombres_clases[np.argmax(prediccion[0])])
print(f'  Liviano: {prediccion[0][0]*100:.1f}%')
print(f'  Pesado:   {prediccion[0][1]*100:.1f}%')
```

6. Registro de Avances del Proyecto

6.1 Hitos Completados en esta evidencia

- Carga y filtrado de CIFAR-10 con tensorflow_datasets: clases automobiline (1) y truck (9) extraídas y re-etiquetadas en esquema binario {0=Liviano, 1=Pesado}.
- Normalización de píxeles [0,255] → [0.0,1.0] implementada como función de mapeo del pipeline tf.data.
- Visualización exploratoria: imagen individual con colorbar + grilla 5×5 con etiquetas.
- Pipeline de datos completo: filter → map(normalizar) → cache → shuffle → batch → repeat.
- Modelo MLP baseline funcional: Input(32,32,3) → Flatten → Dense(128,ReLU) × 2 → Dense(2,Softmax).
- Compilación con Adam + SparseCategoricalCrossentropy y entrenamiento por 10 épocas.
- Curvas de aprendizaje (loss y accuracy por época) graficadas.
- Evaluación formal sobre datos de prueba con reporte de pérdida y accuracy.
- Función de inferencia sobre imagen individual con reporte de probabilidades por clase.

6.2 Próximos Pasos (Evidencia 3)

#	Tarea pendiente	Prioridad / Fundamento
1	Implementar arquitectura CNN: reemplazar el modelo por capas Conv2D + MaxPooling + Dropout para preservar y explotar la estructura espacial de las imágenes.	ALTA — objetivo central del proyecto
2	Aplicar operadores: Sobel, Canny y/o Laplaciano sobre las imágenes para	ALTA

	análisis de bordes y evaluación de su impacto como canal adicional de entrada.	
3	Incorporar Data Augmentation: RandomFlip, RandomRotation, RandomZoom en el pipeline de entrenamiento para mejorar la generalización del modelo.	MEDIA — mejora robustez, planificada en metodología
4	Implementar métricas completas: precision, recall, F1-score por clase y matriz de confusión con foco en la clase Pesado (truck) — objetivo declarado del proyecto.	MEDIA — necesario para evaluación formal
5	Agregar conjunto de validación explícito: separar un split de validación (15%) del entrenamiento para monitorear overfitting durante el entrenamiento con early stopping.	MEDIA — buena práctica para evitar sobreajuste
6	Calcular indicador de proporción logística: aplicar el modelo entrenado sobre el set de test completo y calcular % de vehículos Pesados como KPI del proyecto.	MEDIA — requiere modelo CNN entrenado

7. Tecnologías y Herramientas Utilizadas

Tecnología / Librería	Versión	Uso en el proyecto
Python	3.x	Lenguaje principal del proyecto
TensorFlow / Keras	2.x	Pipeline tf.data, capas Sequential, entrenamiento, evaluación e inferencia
tensorflow_datasets	tfds	Carga estandarizada de CIFAR-10 con splits train/test
NumPy	—	Manipulación de arrays de imágenes y etiquetas para inferencia
Matplotlib	—	Visualización exploratoria, curvas de entrenamiento, grilla de predicciones
Google Colab	—	Entorno de ejecución con GPU T4, acceso al notebook ev2_veh_classifier.ipynb
OpenCV (planificado)	cv2	<i>Próxima evidencia: operadores Sobel, Canny y Laplaciano sobre imágenes del dataset</i>